The Agent Identity Manager (AIM): Accumulating Trust for AI Agent Authentication

Authors Anonymized

***Abstract*—The rapid spread of autonomous AI agents based on large language models (LLMs) has highlighted the lack of an infrastructure that can reliably authenticate these agents. Existing authentication mechanisms (API keys, OAuth, JWT) depend on central servers, carry single point of failure risk and do not provide a standard mechanism for agent-to-agent authentication. Cryptographic accumulators, on the other hand, are a mature cryptographic primitive for set membership verification but have not yet been applied to AI agent authentication. In this work, we propose the Agent Identity Manager (AIM) framework, which uses a multiple cuckoo filter-based cryptographic accumulator architecture. AIM is supported by a dedicated application layer protocol (AIM Protocol) that standardizes agent registration, verification, revocation and synchronization operations. Accumulating the same agent identities into multiple filters with different hash functions reduces the combined false positive rate. The inherent false positive risk of this data structure is managed through a two-stage verification strategy (fast local check and central exact verification for critical operations). This work presents the first system architecture applying symmetric accumulator data structures to AI agent authentication, with feasibility evaluations demonstrating its practical viability and establishing a foundational starting point for future research.**

Keywords— AI agent authentication, cryptographic accumulator, cuckoo filter, multi-agent systems

I. Introduction

Autonomous agents based on large language models (LLMs) are expanding rapidly across software development, customer service, financial analysis, and scientific research. Frameworks such as LangChain [1], AutoGen [2], and CrewAI [3] facilitate multi-agent coordination, accelerating industrial adoption. However, this growth reveals a critical infrastructure gap: existing systems lack mechanisms to reliably verify agent identity, manage authorizations, or immediately revoke access.

Industry metrics highlight the scale of this security deficit. Gartner projects that enterprise applications incorporating task-specific AI agents will rise from under 5% in 2025 to 40% by late 2026 [4]. Cloud Security Alliance surveys further report that 68% of organizations cannot distinguish agent activity from human activity, 74% over-privilege agents [5], only 21% maintain real-time agent inventories, and 84% expect to fail agent-focused compliance audits [6]. Consequently, agent identity management has become an urgent operational imperative.

Architectural limitations in traditional authentication mechanisms underpin this challenge. Standard protocols including static API keys, OAuth, and JSON Web Tokens were designed for human users and conventional software services. Their reliance on central authentication authorities introduces single points of failure and scalability bottlenecks in expanding agent populations. Crucially, these systems lack decentralized standards for peer-to-peer agent authentication.

Cryptographic accumulators offer a promising alternative by enabling compact set membership verification. Originally introduced by Benaloh and De Mare [7], variants such as RSA accumulators, Merkle trees, and bilinear maps have been applied to certificate revocation [8] and anonymous authentication [9], yet remain unexplored in AI agent authentication. Standard centralized protocols such as LDAP [10], OpenID Connect [11], and RADIUS [12] each face well-documented scalability or security limitations at scale. To address this, Çekiş et al. [13] proposed an accumulator-based edge protocol offering O(1) authentication complexity. Additionally, Medury et al. [14] utilized cascading cuckoo filter algorithms to eliminate false positives in deterministic sets.

The Agent Identity Manager (AIM) framework extends accumulator-based distributed authentication [13] to AI agents by deploying a multi-cuckoo filter architecture [14]. AIM balances a central identity authority with agent-side filter replicas, accumulating identical agent identities across k cuckoo filters [15] using pairwise independent hash functions. Assuming independent and identically distributed filters, this multi-filter design multiplicatively reduces the false positive rate from a single-filter rate of ε to ε^k. For high-security operations, secondary verification against the central Agent Registry eliminates unauthorized access from residual false positives. Furthermore, local filter replicas enable agents to execute registration and revocation queries with zero network round-trips, reducing central reliance.

Decentralized verification provides three core structural benefits. First, it improves availability by allowing inter-agent authentication during network partitioning or central authority outages. Second, it expands system capacity by eliminating the throughput ceiling inherent to centralized state queries under high concurrent loads. Third, it reduces per-interaction latency, removing the need for sampling-based verification in multi-agent workflows.

AIM standardizes registration, verification, revocation, and filter synchronization through a custom application-layer protocol. This study contributes a multi-cuckoo filter accumulator architecture tailored for AI agent authentication, a request-signing verification workflow using public key fingerprints, and empirical feasibility and conformance evaluations of the system's critical components.

The remainder of this paper is organized as follows: Section II reviews prior literature; Section III introduces cuckoo filters and cryptographic accumulators; Section IV details the technical specifications of the AIM framework and its protocol; Section V describes the experimental setup and results of the feasibility tests. Finally, Section VI evaluates the findings, while Section VII establishes the trajectory for future research.

II. Related Works

This section reviews state-of-the-art research across three main domains related to the proposed work, covering AI agent

identity management, cryptographic accumulator authentication, and cuckoo filter applications in security. It subsequently presents the literature gap at the intersection of these fields.

Since 2024, AI agent identity management has rapidly progressed across academia and industry. Chan et al. examined agent identifiers, real-time monitoring, and activity logging to enhance visibility [16], while proposing a conceptual framework for assigning identities to AI instances [17]. Cryptographically, Rodriguez Garzon et al. [18] introduced self-sovereign identity using W3C Decentralized Identifiers and Verifiable Credentials integrated with LangChain and AutoGen, though highlighting the limitations of self-managed security in language models. AgentCrypt [19] introduced a four-layer homomorphic encryption framework for agent-to-agent communication. Industry developments include the OWASP Agentic AI Top 10 risk taxonomy [20], a NIST project on software and AI agent identity [21], and Microsoft's Agent Governance Toolkit featuring Ed25519 identities and the Inter-Agent Trust Protocol [22], though the latter still relies on traditional Public Key Infrastructure primitives.

Cryptographic accumulators offer a lightweight, distributed alternative to traditional public key infrastructures by providing compact set-membership verification. First introduced by Benaloh and De Mare [7], these structures evolved through dynamic accumulators designed by Camenisch and Lysyanskaya [9], which enable constant-cost updates and zero-knowledge verification. Subsequent research categorized accumulator evolution [23], analyzed symmetric and asymmetric complexities [24], and evaluated blockchain integration [25]. Practical authentication applications include RSA accumulators for distributed certificate revocation [8] and dynamic accumulators for federated learning [26]. Recently, Çekiş et al. [13] introduced an elliptic curve dynamic asymmetric accumulator protocol for O(1)-complexity edge authentication. However, their design incurs witness management overhead and does not target AI agents. In contrast, the proposed AIM framework utilizes a symmetric accumulator in the form of a cuckoo filter, eliminating witness management costs, supporting deletions, and multiplicatively reducing false-positive rates through a multi-filter architecture.

Cuckoo filters offer scalable, low-latency performance as practical symmetric alternatives to Bloom filters [15]. Their authentication applications have focused predominantly on vehicular networks, including privacy-preserving schemes [27], pseudonym certificate revocation [28], and 5G-AKA batch authentication [29]. In healthcare, Moni and Gupta [30] implemented a dual positive-negative cuckoo filter architecture for anonymous mutual authentication in patient monitoring networks. However, these deployments remain limited to traditional domains such as vehicular networks, 5G, and the Internet of Medical Things.

Existing AI agent identity solutions, including decentralized identifiers with verifiable credentials [18], zero-knowledge virtual machines, and OAuth extensions, deliver deterministic verification but depend on centralized components for validity and revocation checks. Outages in these central authorities halt verification, while increasing agent counts create performance bottlenecks. Although cuckoo filters have proven effective in vehicular and medical networks, no prior studies apply cryptographic accumulators or probabilistic data structures to AI agent authentication. The present study addresses this research gap.

III. Background

This section outlines the general concept of cryptographic accumulators alongside their specific implementation as cuckoo filters.

A. Cryptographic Accumulators

A cryptographic accumulator represents a set $A = \{y_1, y_2, ..., y_n\}$ with a compact, fixed-size value z to enable efficient membership verification [24], [31]. Accumulators are broadly categorized into symmetric and asymmetric classes based on their underlying cryptographic primitives [24]. Symmetric accumulators, such as Bloom filters [33] and cuckoo filters [15], store element fingerprints using k hash functions to support O(1) direct membership queries without witness generation [32]. While computationally efficient, these structures can yield false positives. In contrast, asymmetric accumulators, including RSA-based schemes [7], utilize quasi-commutative hash functions and asymmetric primitives [7]. They evaluate membership deterministically via a function $\mathrm{Ver}(k, z, y_i, w_i) \rightarrow \{\mathrm{TRUE}, \mathrm{FALSE}\}$ using per-element witness values w_i [31], which eliminates false positives at the cost of higher generation and maintenance overhead.

Accumulators are also distinguished by operational functionality and proof type. Dynamic accumulators permit efficient element insertions and deletions, whereas static structures do not [9]. Regarding proof scope, positive accumulators establish membership, negative accumulators establish non-membership, and universal accumulators support both functions [24]. This study adopts cuckoo filters, selecting them from the dynamic, positive, symmetric accumulator class.

B. Cuckoo Filters

The cuckoo filter is a probabilistic data structure proposed by Fan et al. [15] in 2014 that performs approximate membership queries. Based on cuckoo hashing, the structure stores an f-bit fingerprint for each element within one of two candidate buckets [15]. When a primary bucket is full during insertion, the existing entry is displaced to its secondary bucket, optimizing space efficiency [24]. Queries inspect both candidate buckets; the absence of a fingerprint confirms non-membership, whereas a match indicates probable membership. A filter's false positive rate depends on the fingerprint length f and bucket capacity b. This rate is approximately expressed as [15]:

$$\varepsilon \approx 2b/2^f \tag{1}$$

Fan et al. demonstrated that cuckoo filters achieve lower false positive rates and greater space efficiency than space-optimized Bloom filters for a given capacity [15]. Furthermore, whereas Bloom filters are static accumulators that require a complete rebuild to delete elements due to shared bit positions, cuckoo filters support dynamic element deletion [24], [33].

The proposed multi-filter scheme inserts identical elements into k independent cuckoo filters using distinct hash functions. Requiring a positive match across all k filters yields a combined false positive rate of:

$$\varepsilon_{\text{combined}} \approx \varepsilon^{k} \quad (2)$$

This multiplicative reduction rests on the assumption that the false positive events of the filters are independent and identically distributed (i.i.d.) for independent events, the joint probability is equal to the product of their individual probabilities.

IV. System Design

A. Working Principle

At the core of the AIM framework lies a multiple cuckoo filter-based accumulator structure. The working principle of this structure is shown in Fig. 1.

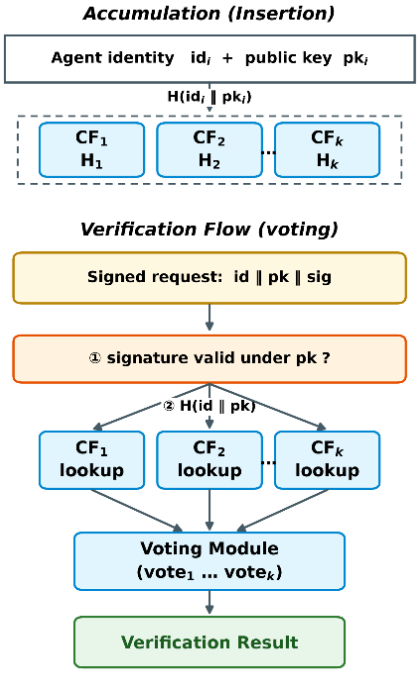

Fig. 1. Working principle of the multiple cuckoo filter-based accumulator.

Each registered agent ID is inserted into k independent cuckoo filters using distinct hash functions(H_1, H_2, ..., H_k), with verification determined by their combined output. This multi-filter architecture exponentially reduces the false positive rate to ε^k, where ε represents the single-filter error rate. Crucially, unlike Bloom filters [33], cuckoo filters natively support element deletion [15], [24]. In agent revocation scenarios, this structural feature enables O(1) removal of a revoked agent's ID across all filters without requiring filter reconstruction.

B. System Components

The AIM framework consists of four main components. These components and the interactions between them are shown in Fig. 2.

AIM Server: This is the identity authority. It is responsible for agent ID generation, the management of master cuckoo filters (k filters) and the maintenance of the Agent Registry. All agent registration, revocation and filter synchronization operations are coordinated through this component.

Master Filters: These are k redundant cuckoo filters located on the AIM Server. Each one accumulates the same agent ID set using different hash functions. These filters are updated during registration and revocation operations, and their updated copies are distributed to all AIM Clients.

Agent Registry: This is the complete and exact database of all agent IDs. Since cuckoo filters are probabilistic structures, the registry is used as a fallback source in situations where exact verification is required (critical operations).

AIM Client: This is the client module embedded in each agent's runtime environment. It maintains local copies of master filters and uses internal accumulators for rapid, offline verification. For critical operations or insufficient local data, the client queries the central AIM Server's Agent Registry for exact verification, mitigating the risks inherent to its probabilistic structure.

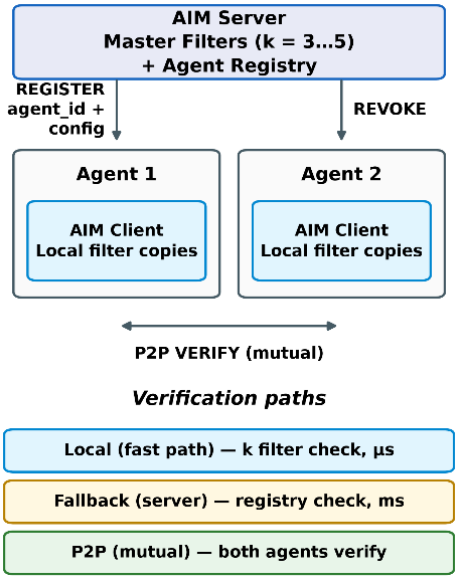

Fig. 2. AIM framework components and their connections.

C. System Workflow

The system workflow can be examined in two main phases: registration and verification. This flow is shown as a sequence diagram in Fig. 3.

1) Registration Flow

When a new agent is registered, it transmits a REGISTER request alongside its metadata to the AIM server. The server validates the request, generates a unique identifier, and records it in the Agent Registry. It then inserts this identifier into all k master filters utilizing distinct hash functions. Subsequently, the server returns the required credentials and configuration parameters, enabling the agent to participate in verification procedures. Finally, the updated filters are distributed to connected clients via a synchronization mechanism to ensure system-wide consistency among local copies.

2) Verification Flow

Local Verification (Flow 1): To authenticate its identity, the agent signs its request using its private key and appends its agent_id and public key to the request. The verifying agent's AIM Client first validates the signature against the provided public key to establish that the request originated from the key holder. Subsequently, it searches across all k local filters for the fingerprint of the identity and public key. An acceptance decision is generated if and only if the signature is valid and

all filters cast an affirmative vote; a single negative vote from any filter suffices to reject the request. Because the workflow requires no network communication, registration and revocation status queries are completed without consulting a central authority.

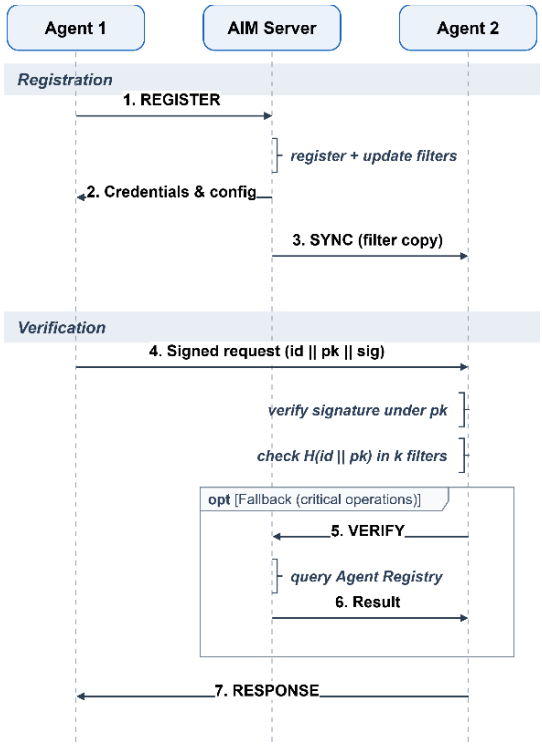

Fig. 3. AIM framework workflow.

Central Fallback (Flow 2): In critical operations (sensitive data access, highly privileged resources) or in situations where local verification may be insufficient, the AIM Client sends a VERIFY message to the central AIM Server. The AIM Server performs an exact check against the Agent Registry and completes the verification with a challenge-response. This fallback mechanism makes it possible to manage the risk inherent to the probabilistic structure and to ensure reliability in scenarios that require exact verification.

An acceptance decision relies on the joint vote of k filters. The transition of operations to the centralized path is governed by an application-configurable threshold based on the inherent risk of the operation. In the proposed design, operations are categorized into three classes: message ingestion and read-only calls are validated exclusively locally; operations that modify shared state are confirmed through centralized validation; and irreversible or high-privilege operations (such as credential access, data export, and financial transactions) are centrally validated under all conditions. Incorporating per-filter trust weights into the voting process has been deferred to future work.

D. AIM Protocol

The AIM Protocol is an application layer protocol that standardizes all communication between the server and clients. Operating over TCP sockets, each message consists of a JSON payload framed by a length prefix. The four primary operational messages are REGISTER, VERIFY, REVOKE, and SYNC; these are accompanied by SUBSCRIBE and INVALIDATE for synchronization subscriptions, CHALLENGE and RESPONSE for identity authentication, and ERROR for error reporting. During centralized authentication, the server generates a single-use nonce, and the agent proves possession of its private key by signing this nonce. Filter synchronization is achieved via a versioned pull mechanism: the client reports its current version, and the server transmits a state snapshot only when the client is out of date. Following revocation operations, the server triggers an update by broadcasting an INVALIDATE message containing only the new version number to subscribed clients. Finally, Transport Layer Security (TLS) serves as an optional security extension that can be integrated into the transport layer.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup and Methodology

The proposed architecture was evaluated through independent feasibility and compliance tests structured around reproducibility, decoupled measurement, and baseline comparison. To ensure reproducibility, each test executes with a fixed master seed and logs all operational parameters. Because joint false-positive events under strong fingerprints are unobservable within feasible query volumes, measurement configurations were decoupled from operational settings, enabling metrics to be gathered on a degraded setup and structurally extrapolated. Identical workloads were also evaluated against a baseline centralized identity authority performing standard JWT verification with a centralized denylist, reflecting the tradeoff between immediate revocation and statelessness. Executed in-process on a single machine to isolate computational overhead from network latency, this baseline highlights relative scaling behavior rather than absolute performance metrics.

Performance was benchmarked via parameter sweeps across agent population sizes (1,000 to 100,000), filter counts ($k \in \{1, 3, 5\}$), and fingerprint lengths. The operational setup uses a 32-bit fingerprint determined by the target false-positive rate, with bucket counts scaling alongside agent populations to manage addressing. Because credential content does not impact verification overhead, these parameters dictate system performance. Reported local latency includes AND-voting across $k = 3$ filters, while memory overhead is evaluated independently.

B. Hash Independence and Joint False Positive Rate

The framework's claim that stacked filters exponentially reduce false positives relies on mutually independent error events across filters. Because non-independence breaks the multiplication rule and invalidates this core premise, experimentally testing independence is essential. To evaluate this property, three hashing schemes were tested, comprising AIM's independent-key design, a sequential-seed negative control, and a double-hashing method using two base hashes. System behavior was assessed using the ratio of observed to theoretical joint false positive rates, where values near 1 confirm multiplication rule validity, and the pairwise false positive φ-correlation coefficient, where values near 0 verify error independence. Figure 4 presents the complete results.

In AIM's independent setup and double hashing, empirical false positive rates align with theoretical predictions within measurement precision, yielding pairwise φ-coefficients indistinguishable from zero. Single-filter empirical rates also match theoretical expectations within a 10% margin for fingerprints up to 16 bits. Conversely, the consecutive seed

shortcut degrades performance, increasing error rates over thirtyfold and shifting mean φ significantly positive due to correlated filter errors. While three independent filters enhance protection seventy-five-fold relative to a single filter, shortcut filters provide only a two-fold gain, effectively reducing protection to single-filter levels despite full computational costs. This negative control demonstrates that system protection depends on filter independence rather than filter quantity.

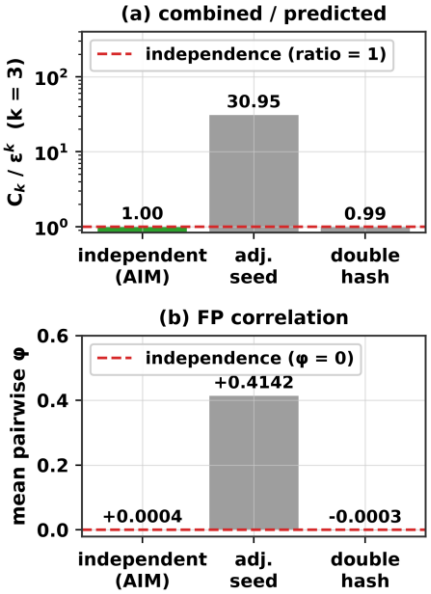

Fig. 4. Comparison of three hash configurations in terms of independence: (a) the ratio of the observed combined false positive rate to the independence prediction (k = 3; expected value 1), (b) the mean φ correlation between filter pairs (expected value 0). The setup on the left is the design used by AIM.

For longer fingerprints, direct empirical measurement is constrained by low event frequencies. Millions of operational queries yielded zero false positive events, as expected rates fell below observation thresholds. Consequently, operational combined rates are reported through extrapolation, whereas filter independence is validated in relaxed, measurable configurations.

C. *Verification Latency and Network Round-Trip Cost*

Two-stage verification risk management requires a low-cost local path, while central authority independence necessitates zero network round trips. This section evaluates each verification path by computational cost and round-trip count, as illustrated in Figure 5 through baseline measurements and total latency projections.

Figure 5(a) details baseline performance. Local AIM and JWT execute in-process, isolating CPU costs for signature verification and revocation from socket overhead. Conversely, central AIM operates over an active network, incurring transport overhead and making network round trips the primary comparative metric. Under this metric, verification requires zero round trips for both JWT and local AIM, compared to two for central AIM. Between the two local approaches, JWT requires a round trip per user for token issuance, while local AIM only incurs round trips during periodic filter updates across all authorized users, resulting in significantly fewer total trips. Central AIM employs a challenge-response authentication scheme to protect the secret key from network exposure. This two-round-trip latency is acceptable because central AIM functions as an infrequent fallback rather than a hot execution path.

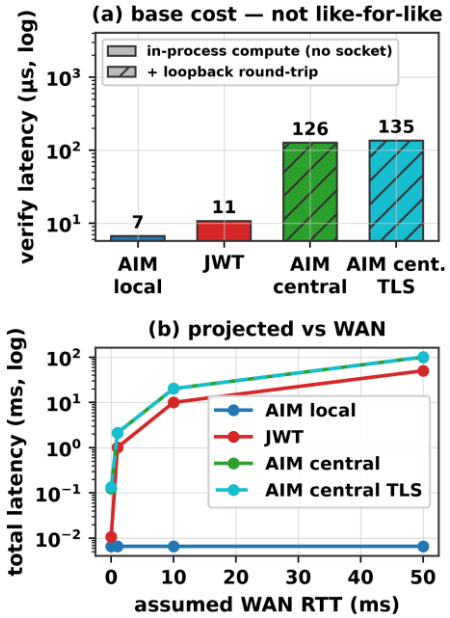

Fig. 5. Verification cost: (a) measured baseline costs; hatched bars include an actual socket round-trip, solid bars represent in-process computation only; (b) total latency projection as a function of the number of round-trips.

Figure 5(b) models latency by adding measured baseline costs to the product of round trips and assumed round-trip time (RTT), assuming linear accumulation without significant bandwidth or packet loss impacts. At a transcontinental 50 ms RTT, microsecond-level local verification remains constant, whereas single- and two-round-trip paths reach tens and hundreds of milliseconds, respectively, causing RTT to dominate computational baseline differences. Furthermore, persistent connections render per-verification TLS overhead negligible by incurring handshake costs only once, yielding nearly identical results for central paths with or without TLS.

D. *Memory Overhead and Resilience of the Multi-Filter Architecture*

Although empirical results validate the framework, running 3 parallel filters incurs a 20% memory overhead relative to an equivalent single filter for short and medium fingerprints. Consequently, the architecture is motivated by decentralized protection and graceful degradation rather than spatial efficiency. Each compromised filter increases attack success by approximately one order of magnitude. Unlike single-filter designs where a single breach causes total system failure, this multi-filter approach forces an adversary to neutralize every independent filter simultaneously.

VI. Discussion and Conclusion

The rapid growth of LLM-driven autonomous agents highlights the lack of reliable authentication infrastructure. Most organizations cannot distinguish agent activity from human actions, causing failures in compliance audits [5], [6]. Conventional mechanisms including static API keys, OAuth, and JWT rely on central servers, introduce single points of failure, and lack standardized agent-to-agent authentication protocols. Although cryptographic accumulators provide compact set-membership proofs, they have not been applied to AI agent authentication. To address this research gap, we propose and empirically evaluate Agent

Identity Manager (AIM), a novel authentication framework. By combining a multi-cuckoo filter architecture, two-stage verification, and distributed filter replicas, AIM reduces false-positive rates to ε^k, minimizes central revocation dependencies, and achieves microsecond local verification without network round-trips under the independence assumption.

Empirical testing reveals inherent structural trade-offs within the architecture. While multi-filtering adds overhead for short fingerprints relative to single filters, deploying multiple short filters increases the overall reliability of the system by eliminating a single point of failure. AIM prioritizes resilience against offline exhaustive searches and graceful degradation over space efficiency. Additionally, two-round challenge-response verification is costlier than single-round token verification, though it remains acceptable as an infrequent fallback mechanism. Because initial evaluations focused on identity primitives rather than live autonomous agents, measuring runtime overhead within complete workflows remains an objective for future research.

Future research will transition AIM into operational multi-agent systems to measure exact runtime verification overhead. A primary security focus involves binding identities to executing agent instances to mitigate impersonation risks from compromised keys. Planned enhancements include implementing runtime attestation via hardware security modules such as trusted platform modules or secure enclaves, deploying short-lived identities, and requiring runtime signatures. Furthermore, expanded testing under realistic workloads will quantify revocation latency by tracking how long revoked agents remain valid before local filter replicas update.

REFERENCES

[1] LangChain, "LangChain: Build context-aware reasoning applications," 2024. [Online]. Available: https://www.langchain.com

[2] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[3] CrewAI, "CrewAI: Framework for orchestrating role-playing autonomous AI agents," 2024. [Online]. Available: https://www.crewai.com

[4] Gartner, Inc., "Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025," Press Release, Aug. 26, 2025.

[5] Cloud Security Alliance, "The identity and access gaps in the age of autonomous AI," Survey Report, commissioned by Aembit, Mar. 2026.

[6] Cloud Security Alliance, "Securing autonomous AI agents," Survey Report, commissioned by Strata Identity, Feb. 2026.

[7] J. Benaloh and M. de Mare, "One-way accumulators: A decentralized alternative to digital signatures," in Proc. EUROCRYPT '93, Springer LNCS, vol. 765, 1993, pp. 274–285.

[8] İ. Özçelik and A. Skjellum, "CryptoRevocate: A cryptographic accumulator based distributed certificate revocation list," in Proc. 11th IEEE CCWC, 2021, pp. 865–872.

[9] J. Camenisch and A. Lysyanskaya, "Dynamic accumulators and application to efficient revocation of anonymous credentials," in Proc. CRYPTO 2002, Springer LNCS, vol. 2442, 2002, pp. 61–76.

[10] V. Koutsonikola and A. Vakali, "LDAP: Framework, practices, and trends," IEEE Internet Comput., vol. 8, no. 5, pp. 66–72, 2004.

[11] D. Fett, R. Küsters, and G. Schmitz, "The web SSO standard OpenID Connect: In-depth formal security analysis and security guidelines," in Proc. IEEE 30th CSF, 2017, pp. 189–202.

[12] S. Goldberg et al., "RADIUS/UDP considered harmful," in Proc. 33rd USENIX Security, 2024, pp. 7429–7446.

[13] İ. K. Çekiş, A. Toros, V. Yiğit, and İ. Özçelik, "Edge authentication using a cryptographic accumulator," in Proc. 2025 9th Int. Symp. Multidisciplinary Studies and Innovative Technologies (ISMSIT), IEEE, 2025, pp. 1–5.

[14] S. Medury, A. Altarawneh, and A. Skjellum, "Design and evaluation of cascading cuckoo filters for zero-false-positive membership services," in Proc. 11th IEEE CCWC, 2021, pp. 1061–1065.

[15] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than Bloom," in Proc. 10th ACM CoNEXT, 2014, pp. 75–88.

[16] A. Chan et al., "Visibility into AI agents," in Proc. ACM FAccT '24, 2024, pp. 958–973.

[17] A. Chan et al., "IDs for AI systems," arXiv:2406.12137, 2024.

[18] S. Rodriguez Garzon et al., "AI agents with decentralized identifiers and verifiable credentials," arXiv:2511.02841, 2025.

[19] H. Karthikeyan et al., "AgentCrypt: Advancing privacy and secure computation in AI agent collaboration," in Proc. NeurIPS Workshop on Scaling Environment for Agents, 2025.

[20] OWASP, "Top 10 for Agentic Applications 2026," Dec. 2025. [Online]. Available: https://genai.owasp.org

[21] R. Galluzzo, B. Fisher, H. Booth, and J. Roberts, "Accelerating the adoption of software and AI agent identity and authorization," NIST NCCoE Concept Paper, Feb. 2026.

[22] Microsoft, "Agent Governance Toolkit," GitHub, MIT License, Apr. 2026. [Online]. Available: https://github.com/microsoft/agent-governance-toolkit

[23] Y. Ren, X. Liu, Q. Wu, L. Wang, and W. Zhang, "Cryptographic accumulator and its application: A survey," Security and Communication Networks, vol. 2022, Article 5429195, 2022.

[24] İ. Özçelik, S. Medury, J. Broaddus, and A. Skjellum, "An overview of cryptographic accumulators," in Proc. 7th Int. Conf. Information Systems Security and Privacy (ICISSP), 2021, pp. 661–669.

[25] M. Loporchio, A. Bernasconi, D. Di Francesco Maesa, and L. Ricci, "A survey of set accumulators for blockchain systems," Computer Science Review, vol. 49, Article 100570, 2023.

[26] S. Li, Y. Zhang, J. Bai, and K. Fan, "DA³4FL: A robust dynamic accumulator-based authentication and key agreement with preserving model training data integrity for federated learning," Nature Scientific Reports, Article s41598-025-33685-1, 2025.

[27] J. Cui, J. Zhang, H. Zhong, and Y. Xu, "SPACF: A secure privacy-preserving authentication scheme for VANET with cuckoo filter," IEEE Trans. Veh. Technol., vol. 66, no. 11, pp. 10283–10295, 2017.

[28] H. Zhang, D. Zhang, H. Chen, and J. Xu, "Improving efficiency of pseudonym revocation in VANET using cuckoo filter," in Proc. 20th IEEE Int. Conf. Communication Technology (ICCT), 2020, pp. 763–769.

[29] C. Kalalas and J. Alonso-Zarate, "Lightweight and space-efficient vehicle authentication based on cuckoo filter," in Proc. IEEE 5G World Forum (5GWF), 2020, pp. 139–144.

[30] S. S. Moni and D. Gupta, "Secure and efficient privacy-preserving authentication scheme using cuckoo filter in remote patient monitoring network," in Proc. IEEE TPS-ISA, 2022, pp. 208–216.

[31] N. Fazio and A. Nicolosi, "Cryptographic accumulators: Definitions, constructions and applications," Technical Report, New York University, 2002.

[32] A. Kumar, P. Lafourcade, and C. Lauradoux, "Performances of cryptographic accumulators," in Proc. 39th Annual IEEE Conf. Local Computer Networks, 2014, pp. 366–369.

[33] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," Communications of the ACM, vol. 13, no. 7, pp. 422–426, 1970.